

TP1 Pandas

Objectifs du TP : Pratiquer les opérations essentielles de Pandas sur des données géographiques réelles : deux fichiers CSV représentant des pays et des villes du monde entier.

- Charger et explorer un DataFrame
- Accéder aux données (iloc, loc, sélection de colonnes)
- Détecter et traiter les valeurs manquantes (NaN)
- Créer des colonnes calculées
- Trier et filtrer des données
- Fusionner deux DataFrames (merge)

Préambule :

- Télécharger depuis Moodle et dans votre répertoire de travail les deux fichiers **countries.csv** (251 lignes) et **cities.csv** (49586 lignes). Les deux fichiers utilisent le point-virgule (;) comme séparateur. Les fichiers contiennent des valeurs manquantes (NaN) réelles. Certaines questions portent spécifiquement sur leur détection et traitement.

Travail demandé :

- Créer un nouveau programme Python TP1.py qui effectue les opérations qui suivent. Toutes les opérations n'ont pas été nécessairement vues en cours, il faudra donc vous aider de la documentation en ligne (pandas.pydata.org/docs).
- Importez les packages pandas et numpy (les installer si nécessaire)

Partie 1 - Exploration

Q1 Chargement et dimensions

- Lire chaque fichier **.csv** dans un dataframe (nommés **pays** et **villes**)
- Afficher (ce sera partiellement car les fichiers sont volumineux) ces deux dataframes ainsi que leur nombre de lignes et de colonnes.

Q2 Exploration du DataFrame villes

Sur le DataFrame villes, afficher :

- Les 3 premières lignes
- Un échantillon aléatoire de 10 lignes
- Le nom des colonnes
- Le type de données de chaque colonne

Q3 Statistiques descriptives

- Appliquez la méthode `describe()` sur le DataFrame `villes`. Que constatez-vous sur la colonne `population` ? Quelle est la population minimale observée ? `pop min = 0`

Q4 Sélection des colonnes

- Afficher seulement le nom des villes, leur latitude et leur longitude pour les 10 premières lignes du DataFrame `villes`.

Q5 Accès par position : `iloc`

- Afficher toutes les informations de la ligne 11 (index 10) du DataFrame `villes`, puis récupérez uniquement son nom de ville dans une variable.

Q6 Type d'une Series

- Récupérer dans une variable `data` la liste de tous les noms de villes (colonne `name`). Affichez son type Python. Quelle est la différence avec une liste classique ? `classe str et non list`

Partie 2 - Valeurs manquantes

Q7 Détecter les valeurs manquantes

Sur le DataFrame `pays`, afficher :

- Le nombre de NaN par colonne
- Le pourcentage de NaN par colonne
- Les lignes qui contiennent au moins un NaN

Q8 Traiter les NaN

Effectuer les opérations suivantes sur le DataFrame `pays` :

- Comptez combien de lignes seraient supprimées par `dropna()`
- Remplacez les NaN de la colonne `continent` par la valeur 'Inconnu'
- Remplacez les NaN de `currency_code` et `currency_name` par 'N/A'
- Vérifiez qu'il ne reste plus de NaN dans ces colonnes

Q9 Villes sans nom et villes sans pays

Le DataFrame `villes` contient également des NaN :

- Combien de villes n'ont pas de nom (colonne `name` est NaN) ? `1`
- Combien de villes n'ont pas de pays associé (colonne `country` est NaN) ? `35`
- Affichez ces lignes problématiques
- Supprimez les lignes sans nom de ville avec `dropna(subset=['name'])`

Résultat attendu : 1 ville sans nom, 35 villes sans pays associé

Partie 3 - Valeurs calculées

Q10 Densité de population

Créer une nouvelle colonne `density` dans le DataFrame `pays`, calculée comme `population / area`. Afficher ensuite les 10 pays les plus denses.

Q11 Catégorie de population

Créer une colonne `pop_category` dans le DataFrame `villes` qui classe chaque ville selon sa population :

- `population < 10 000` => petite ville

- $10\,000 \leq \text{population} < 100\,000 \Rightarrow$ ville moyenne
- $100\,000 \leq \text{population} < 1\,000\,000 \Rightarrow$ grande ville
- $\text{population} \geq 1\,000\,000 \Rightarrow$ Métropole

Utilisez `apply()` avec une fonction lambda ou une fonction nommée. Vérifiez avec `value_counts()` la répartition obtenue

Q12 Hémisphère : `np.where`

Utiliser `np.where()` pour créer une colonne `hemisphere` dans le DataFrame `villes`, avec la valeur 'Nord' si `latitude >= 0`, 'Sud' sinon.

Afficher ensuite le nombre de villes par hémisphère avec `value_counts()`.

Q13 Tranches de surface : `pd.cut`

Utiliser `pd.cut()` pour créer une colonne `taille` dans le DataFrame `pays`, qui classe chaque pays selon sa surface :

- 0 - 10 000 : Micro-état
- 10 000 - 500 000 : petit pays
- 500 000 - 3 000 000 : Pays moyen
- 3 000 000 - 20 000 000 : Grand pays

Afficher la répartition avec `value_counts()`

Partie 4 - Tri et Filtrage

Q14 Filtrage : pays en euros

Afficher la liste des pays dont la monnaie est l'Euro (`currency_code == 'EUR'`). Combien y en a-t-il ? 35

Q15 Filtrage : codes Dollar sans doublons

Donner la liste des codes de monnaies (`currency_code`) dont le nom contient "Dollar", sans doublons.

Q16 `nlargest` et `nsmallest`

Utiliser `nlargest` et `nsmallest` pour répondre aux questions suivantes :

- Les 2 pays avec la plus grande surface (colonne `area`)
- Les 3 pays avec la plus petite population (colonne `population`)
- Les 10 plus grandes villes par population (colonnes `name`, `population`, `country`)

Résultats attendus : Q16a = Russia (17 100 000 km²), Antarctica (14 000 000 km²), Q16b = Antarctica, Bouvet Island, Heard Island and McDonald Islands (pop. = 0), Q16c = Shanghai, Istanbul, Buenos Aires, Mumbai, Mexico City, Beijing, Karachi, Tianjin, Guangzhou, Delhi

Q17 Tri multi-critères

Trier le DataFrame `pays` par continent (ordre croissant) puis par population (ordre décroissant) en cas d'égalité. Afficher le nom, continent et population des 15 premières lignes.

Q18 Renommer des colonnes

Trier le DataFrame `pays` par continent (ordre croissant) puis par population (ordre décroissant) en cas d'égalité. Afficher le nom, continent et population des 15 premières lignes.

Renommer dans le DataFrame `pays` :

- `name` → `pays_nom`
- `area` → `superficie`

- `population` → `nb_habitants`

Vérifier avec `df.columns` que le renommage a bien été effectué.

Partie 5 - Fusion des dataframes

Q19 Fusionner pays et villes (capitales)

Dans le DataFrame `pays`, la colonne `capital` contient un identifiant (numérique) correspondant à la colonne `id` de villes. Associer chaque pays à sa capitale en 3 étapes :

- Étape 1 : Renommer
Renommez la colonne `name` de villes en `capital_name` pour éviter une collision lors de la fusion.
- Étape 2 : Sélectionner les colonnes utiles
Limiter chaque DataFrame aux colonnes nécessaires avant la fusion pour un résultat lisible.
- Étape 3 : Fusionner avec `merge()`
Afficher les 10 premières lignes du DataFrame fusionné.

Q20 Analyse finale sur le DataFrame fusionné

À partir du DataFrame fusionné (`pays + capitale`), répondre aux questions suivantes :

- Combien de pays n'ont pas de capitale trouvée dans villes (`capital_name` est NaN) ?
- Affichez le nom du pays, sa superficie et le nom de sa capitale pour les 5 pays les plus étendus
- Quelle est la capitale la plus au nord (latitude maximale) ? La plus au sud ?